

CMPS 4191 - Advanced Web Technologies - Test 1 - Week 3 Writeup

2026-09-23 - Reynerio Samos & Chahiim Pop

Image Lab SPA

Async 202 Accepted + Short Polling

Assessment: Build an asynchronous image processing app where an API accepts a JPG or PNG to process it into 3 variants (thumbnail, preview, and display) of varying dimensions. An acknowledgment is sent to the Browser upon image upload. A job is created and stored in a job store, and a worker accepts the job and starts working on image variants. The Browser performs 1-second short polling to check the status of the current job with timestamps, and observes the job state until all 3 image variant previews and downloads are made visible when successful, or the job is marked as finished or fails.

Week 3

We were to implement Short Polling, improve UI state observations, and address cancellation and retrieval errors, with the expectation that the Browser observes and renders the completed job lifecycle for both successful and failed job terminal states.

Work Done:

Fixing UI and DB from Week 2:

For the DB, we changed the primary ID in the `jobs` `images` and `variants` table to UUID v7 and kept the `public_id` that the server sends back to clients as UUID v4.

For the UI, we changed the image upload preview to be larger and moved the image upload information (such as original file name, size, and dimensions) to the bottom of the larger preview. We also made the image variant previews the same size to make them look more consistent.

Week 3

Polling

We changed the poll in `app.js` to start only after a successful `HTTP 202` response is received, NOT on file upload. We also updated to Short polling using `SetTimeout` with a 1-second interval to query the `GET status_url` until a terminal state is reached. Finally, we used the `AbortController` to abort in-flight status requests or when switching jobs to reset UI state (such as on page refresh)

UI

We connected the poll responses to DOM elements to update the status badges and timeline step indicators, and we also included timestamps for each completed/failed step. We also added a section to view the progress of each variant, but this feature needs more work to be implemented correctly. The result grid and generated image variant previews remain hidden until all 3 variants have been generated and the job is marked complete.

Error & Recovery

We had separated the API processing failures from observation/network failures (HTTP failures). We also implemented a try-again mechanism for network/retrieval failures, where the UI retains the current job state, halts polling, and presents the "Try again" button without needing to reupload the image. This feature requires further testing to confirm it is implemented correctly.

We added a new handler and route to implement this recovery feature in `helpers.go` and `routes.go`, with the `POST /v1/jobs/{job_id}/reprocess` endpoint that re-queues failed jobs without requiring reuploads. This will ensure that new jobs created on the same image use the same `image_id` and the stored file reference in the DB records, rather than uploading a new image row to the images table. This feature is not working as intended because of the UUIDs in the databases. Currently, each new job generates a unique, random ID, so further refactoring and testing are needed to properly implement this feature.`

Challenges

Some challenges we faced this week with this week's progress included implementing the new Error & Recovery features for network/observation failures and implementing the recovery feature to reuse the same image ID while changing the job ID. Currently, the app saves the image upload and creates a new image ID and job ID. Another issue we faced was that our status updates for each variant were separate from the total job completion tracking. Looking back at the weekly briefs for this assignment, we were not really supposed to implement this feature. We still want to add this. To make the feature work as intended, we need to decouple the DB jobs and variant locking to allow status checks on individual variants and restructure how image variants are stored.